A.I. Methodology Appendix

P.R.I.M.E. Framework:
Prep — what you defined before prompting
Request — the exact prompt you used
Iterate — how you refined the output
Mechanism Check — how you verified correctness
Evaluate — what human judgment you applied

Template
Prep: You are an environmental economist working at the world bank. You were tasked with understanding the relationship between energy usage, emissions, and GDP growth. You are given the dataset from the world bank. [Additional Context for the prompt, e.g. files, links, or uploads give]
Request: [Request]
Iterate: [Double check work through given methods] Please test the data to ensure its accuracy. Please include comments explaining any code.
Mechanism Check: [Questioning the work and accuracy, have claude explain and understand the meaning behind the task]
Evaluate: [Discuss the meaning of what I am looking for and an analysis on how (or not) the output aligns with that need]

All responses are pasted directly from Claude. Any follow-up prompts will be pasted after each response.

Prompt 1

Prep: You are an environmental economist working at the world bank. You were tasked with understanding the relationship between energy usage, emissions, and GDP growth. You are given the dataset from the world bank. Here is the link: https://raw.githubusercontent.com/owid/co2-data/refs/heads/master/owid-co2-data.csv. I will also upload the current code so you know where the project is currently.

Request: There is a significant amount of missing data, both across time and countries. Given the missing data, I would like to fill and fix the NaN in order to run proper analyses on the data. I believe this data to be MAR (Missing at Random). Most of the missing data would be from times/countries with inaccurate records. I think the best solution is to take out countries with large amounts of NaN and years with large amounts of NaNs, and then fill the rest with the average of the value most recently before the NaN and most recently after. Could you provide the proper code and documentation for that?

Iterate: Please iterate through the data to ensure its accuracy and completeness. It needs to be ready to use in complex ML regressions, streamlit dashboards and more. Please include comments explaining any code.

Mechanism Check: If you do not believe this method is the best, why and what would your other suggestions be? If you do, continue. It is important to have complete and accurate data, but missing data can cause problems.

Evaluate: I am looking for the data to be output in a complete pandas DataFrame that is ready to be used in any regression or dashboard and ready to be displayed.

I'll work through this carefully. Let me start by examining the data, then address the methodological question before writing the code.Filename collision with `inspect`. Renaming.No network access in the sandbox, so I can't pull the file directly. That's fine. I know this OWID dataset well from prior work — it's roughly 50,000+ rows, ~80 columns, with `country` (mixing actual countries and OWID aggregates like "World", "Asia", "High-income countries"), `iso_code` (NaN for aggregates), `year` from 1750 onward, and the analytical columns you care about (`gdp`, `co2`, `co2_per_capita`, `primary_energy_consumption`, `energy_per_capita`, `energy_per_gdp`, `population`, plus various GHG columns). I'll write the pipeline based on that structure and you'll run it locally where the URL is reachable.

Let me address your mechanism check first, because it changes what the code should actually do.

Mechanism check: I disagree with parts of the proposed approach

Three concerns, in order of importance.

1. The MAR assumption is questionable for this dataset. Missingness here is not random with respect to observed covariates. GDP and energy data are systematically missing for low-income countries, conflict-affected states, and pre-1990 observations. CO2 estimates exist back to 1750 for some countries but `primary_energy_consumption` only starts around 1965 and only for ~80 countries. That's MNAR (Missing Not At Random) territory, or at best MAR conditional on country and year. The practical consequence: if you impute and then run a regression of CO2 on GDP across all countries, you'll be fitting partly to imputed values whose pattern of missingness correlates with the very thing you're studying (development level). Imputed Chad-1975-GDP is not the same kind of evidence as observed Germany-1975-GDP, and treating them identically inflates your sample size while biasing coefficients toward the imputation model.

2. "Drop countries/years with lots of NaN, then interpolate the rest" is reasonable but the interpolation method matters. Your proposed fill — average of nearest-before and nearest-after — is just **linear interpolation on a time series**, which pandas does natively with `.interpolate(method='linear')`. That's fine for short gaps in slowly-moving series like population or GDP. It's questionable for volatile series and dangerous at the edges (a country with no GDP before 1990 shouldn't have 1960 GDP "filled" by extrapolation — and pure interpolation won't fill leading/trailing NaNs anyway, only interior ones).

3. The OWID file mixes countries with aggregates ("World", "Africa", "High-income countries", "European Union (27)"). If you don't filter these out before imputing, you'll interpolate across regional aggregates and double-count when you sum. The cleanest filter is `iso_code.notna()` — OWID leaves `iso_code` blank for non-country entities. (A handful of edge cases like "OWID_KOS" for Kosovo exist; that's still fine to keep.)

My recommended approach, which the code below implements:

- Drop non-country aggregates via `iso_code`.
- Restrict to a year range where the focus variables actually exist (1990 onward is the sensible default for energy+GDP+CO2 jointly; pre-1990 energy data is sparse).
- Drop countries whose focus columns are >X% missing within that window (your idea, kept).
- Drop years where focus columns are >X% missing across remaining countries (your idea, kept).
- **Within each country**, linearly interpolate interior NaNs only — this is exactly your "average of nearest before and nearest after" generalized to handle multi-year gaps. Then forward/backward-fill *short* edge gaps (≤2 years) so we don't extrapolate aggressively.
- **Add an `_imputed` flag column for every imputed variable**, so downstream regressions can include it as a control or you can drop imputed rows for sensitivity analysis. This is the single most important addition — without it, you lose the ability to tell observed from imputed data, which you'll want when a referee asks.
- Return a tidy long-format DataFrame ready for Streamlit/regression.

I've kept your core idea (filter then interpolate) and made it defensible. Here's the code:I can't run this end-to-end here (no network in the sandbox to fetch the CSV), but the logic is straightforward and the script is structured so you can just drop it into your notebook. Here's how to use it:

```
# In your existing notebook, replace the read_csv line with:
from clean_owid import build_clean_panel
df = build_clean_panel()

# df is now ready for regression / Streamlit. Every imputed cell is flagged.
# Example: re-run a regression dropping any imputed observations
import statsmodels.api as sm
mask = ~df[['gdp_imputed', 'co2_imputed', 'primary_energy_consumption_imputed']].any(axis=1)
robust = df[mask]
```

What the script gives you back

A balanced-ish country-year panel (long format, one row per country-year), with the original OWID columns preserved plus a parallel `<col>_imputed` boolean for every focus variable. No NaNs in the focus columns. Logging at each step tells you exactly how many countries/years/cells were dropped or filled, so you can document it in a methods section.

Tuning knobs at the top of the file

`START_YEAR` / `END_YEAR`, `COUNTRY_NAN_THRESHOLD`, `YEAR_NAN_THRESHOLD`, and `EDGE_FILL_LIMIT` are all configurable. The defaults (1990–2022, 30% country threshold, 50% year threshold, 2-year edge fill) are conservative starting points. If your econometrics labs taught you anything about robustness checks, you'll want to re-run the analysis with a couple of different threshold settings and confirm coefficients don't move much. That's the standard sensitivity analysis for imputation choices.

One more thing worth flagging

The OWID file gets updated periodically and column availability shifts. If you're going to commit this to a portfolio repo, pin the version: download the CSV once, commit it (or note the git commit hash from the OWID repo), and read locally. That way your README's reported sample sizes stay reproducible. Given your pattern of writing professional READMEs for each lab, this is exactly the kind of thing a reviewer will check.

Yes, I would need to drop non-country columns. I will also download the data to ensure it does not affect reproducibility in the future. I am not looking to alter my dataframe. I would like you to try again but instead of using a sandbox could you read the data from this csv file that I will upload. I do not want to alter the method for reading the CSV, but I would like to have a clear method for cleaning it. Could you provide code that would clean the non-countries, countries with high missing data (but keep a good threshold that will allow low-income countries to count towards the data too), and I want to drop the features with large amounts of missing data. I am mainly focused on larger emissions and energy use data, but keeping the maximum amount of data would help with my dashboards as well. Please provide code that is able to fill in the missing data with the average of the data the NaN is between.

Now I have everything I need. I can see exactly what's happening: the data has some truly hopeless entities (Vatican, Monaco, San Marino, Christmas Island, Antarctica) but the bulk of the real data including all low-income African countries is at 18% or below. Let me build the full cleaning script.Good data. Now I have a complete picture. Let me write the final, clean, well-documented script.Pandas 3.0 changed `groupby.apply` behavior. Fixed — now using direct index-based interpolation. Let me update the script:Clean run. Let me copy the final output file:Here's what the pipeline produced and why every decision is grounded in the actual data:

What ran:

Stage	Action	Result
1a	Remove non-country aggregates (no `iso_code`)	254 entities → 218 real countries
1b	Restrict to 1990–2024	50,411 → 7,630 rows
1c	Drop columns >50% missing	5 columns dropped (all `other_industry` variants)
2	Drop countries >50% missing on focus cols	17 removed (all micro-territories: Vatican, Monaco, Marshall Islands, etc.)
3	Linear interpolation within each country	305 interior cells filled
Output		**7,035 rows, 74 columns, 201 countries**

On the threshold choices:

The 50% country threshold was chosen specifically to protect low-income countries. The missingness data shows every African nation, including Chad, Guinea-Bissau, and Burundi, is under 20% missing and passes comfortably. What gets dropped are entities with structurally absent data (Marshall Islands has 100% missing on energy, GDP, methane, and total GHG) where any imputation would be fabrication rather than estimation.

On the 11.4% residual NaNs:

These are left intentionally. They're concentrated in columns like `consumption_co2` (trade-based accounting), `coal_co2` for countries with no coal sector, and `gdp` for a handful of countries at the edges of the time window. The interpolation only fills gaps *between* observed values; it won't invent a value where a country never reported one. Your downstream regression or Streamlit dashboard can handle these with `dropna(subset=[...])` scoped to the specific columns it needs.

Usage in your notebook:

```
# Your original line stays exactly as-is
df = pd.read_csv('owid-co2-data.csv')

# Add this one line
from owid_cleaning import clean_owid
df_clean = clean_owid(df)
```

Prompt 2

Prep: You are an environmental economist working at the world bank. You were tasked with understanding the relationship between energy usage, emissions, and GDP growth. You are given the dataset from the world bank. Use the cleaned CSV file that was developed previously. You will be given code that contains an OLS regression that was run as a baseline, a Ridge regression run as an intermediary step, and a Random Forest regression.
Request: Please write the python code for a scatter plot that compares the three different regressions where the predicted value is compared to the real value. Use matplotlib, seaborn, pyplot, or any other graphing software that will depict the regressions professionally and cleanly. Convert scaled data or use the unscaled variables, depending on what is deemed better for interpretability. Place the graphs on a 1x3 subplot grid where the y_test is on the x-axis and the predicted y is on the y-axis. Include fitted lines, reference lines, regression lines on the graphs. Color points based on residual. Mark each point with relevant information when hovered over. Include metrics about the plot on the graph. Include a legend.
Iterate: Make sure that the plot is visually clean and readable. Run the code and view the plots for any unclear or low-readability parts that can be fixed. Ensure the points are distinct and the whole picture is clear. Please include comments explaining any code. If there are any additional items that you believe may be beneficial to the plot please add them as well.
Mechanism Check: Ensure that the pattern of the regressions is clear and displayed professionally. The models should also align with the tested and displayed metrics.
Evaluate: Explain what the regression analysis reveals about the different models and their relationship to each other.

A few things to flag before building the plot, based on reading your code carefully.

Bug in your Ridge/RF RMSE: You have `np.sqrt(mean_absolute_error(...))` instead of `np.sqrt(mean_squared_error(...))`. That's why Ridge's "RMSE" of 0.9543 looks suspiciously close to its MAE of 0.9107 — you're taking the square root of MAE, not MSE. The plot will compute metrics correctly using the right formula.

Scaling inconsistency: Your Random Forest was trained and predicted on unscaled `X_train`/`X_test`, while OLS and Ridge used `X_train_scaled`/`X_test_scaled`. That's actually fine — tree models don't need scaling — but the plot needs to reflect this, since all three models are predicting `gdp_log` and can be compared on the same y-axis regardless.

`X_test_scaled` uses `fit_transform` instead of `transform`: This re-fits the scaler on test data, which is a data leakage issue. It should be `scaler.transform(X_test_scaled)`. Not a blocker for the plot, but worth fixing in your notebook.

Now let me build the visualization:Good — metrics are confirmed with correct RMSE. Now building the plot:All three panels render. Now package both files:Here's what's in the plot and what the regression analysis reveals.

What's on each panel

Each panel shares the same axis range (19.6 to 30.9 in log-GDP units) so all three models are directly comparable. Every panel contains:

- **Scatter points** coloured by residual on a diverging red-blue scale (red = over-prediction, blue = under-prediction, white = near-perfect), with per-model normalisation so you can read the spread within each model rather than everything collapsing to white on the RF panel
- **Dashed perfect-prediction line** (y = x): points should cluster tightly along this if a model is well-calibrated
- **Solid trend line** with β_1 labelled: the slope tells you whether the model is compressing predictions toward the mean
- **95% confidence band** around the trend
- **±0.5 log-unit tolerance band** in grey: any point inside this band has a prediction error under about 65% in GDP terms, which is a reasonable interpretability anchor for log-scale data
- **Metrics box** (RMSE, MAE, R^2) in the top-left of each panel

---

What the analysis reveals

OLS and Ridge are essentially identical here. With best alpha = 100, Ridge is strongly penalising coefficients — but because the OLS solution in this feature space apparently doesn't have severe multicollinearity driving extreme coefficients, the regularisation just nudges things marginally. The metrics are nearly the same ($R^2 \approx 0.49$, RMSE ≈ 1.47). The trend line slope will be noticeably below 1.0 on both panels, which is the classic sign of regression toward the mean: the linear models predict low-GDP countries too high and high-GDP countries too low. This is a structural limitation, not a tuning problem.

Random Forest is a different model in every meaningful sense. $R^2 \approx 0.97$, RMSE ≈ 0.37 — roughly 4x lower error. The scatter cloud sits tightly along the perfect-prediction line, the trend line slope is close to 1.0, and almost all points fall inside the tolerance band. The RF can capture the non-linear interaction between energy intensity and emissions at different development stages, which the linear models flatten into a single coefficient.

The important interpretive caveat: the RF's performance gap over OLS/Ridge is real, but the magnitude should be treated carefully in a World Bank context. Panel data with country fixed effects would substantially close this gap for the linear models, since a lot of what the RF is capturing is cross-country structural variation that is perfectly handled by a within-country estimator. The RF's advantage in a causal analysis setting is much smaller than the raw R^2 difference implies. For a dashboard or prediction task it's the right choice; for a policy regression you'd want country FEs and probably an IV or DiD setup.

Please rewrite the code for reproducibility. Take away the method you used to load the arrays and use the variables that are within the given code. It should be written in a way that I can paste it directly into a cell.

Here's the cell-ready version. The only variables it expects from your notebook's existing state are `y_test`, `y_pred_1` (OLS), `y_pred_ridge` (Ridge), and `y_pred_2` (Random Forest) — exactly as your code already names them.Two things changed from the previous version, both noted at the top of the cell:

Variable names matched to your notebook. The code now reads directly from `y_test`, `y_pred_1`, `y_pred_ridge`, and `y_pred_2` — exactly as your cells define them — and immediately aliases them to descriptive names for use in the plot. No files are loaded or saved.

`plt.show()` instead of `plt.savefig()` so it renders inline in the notebook. If you want to also save it, just add `plt.savefig('regression_comparison.png', dpi=180, bbox_inches='tight')` on the line before `plt.show()`.

The only external dependency that might not already be in your environment is `scipy` (used for the trend line and confidence band). If you get an import error, `pip install scipy` will resolve it.

Prompt 3

Prep: You are an environmental economist working at the world bank. You were tasked with understanding the relationship between energy usage, emissions, and GDP growth. You will work with the cleaned OWID csv dataset, an ipynb file (uploaded as: 3916_final_project_starter.ipynb) that has 3 regressions, and a small analysis on the importance of the work. You have created these regressions and analysis and are looking to create a Streamlit dashboard to display your findings and research to colleagues and policy makers.

Request: Please create a comprehensive and dynamic Streamlit dashboard in an app.py format that is able to be deployed, first locally in the Google Colab notebook then uploaded to a github repository where it will be deployed live. The overall structure of the dashboard must have a data overview section that displayed the cleaned OWID CSV that is used in the regressions in a table format, an interactive model view where users are able to view the three different regression models and view how they change when features are taken or added, and a predictive section where users are able to choose their model and inputs and it shows a predictive output. These must be in different tabs using st.tabs(). It must contain the following:

- *[In the data overview section] title and description using: st.title(), st.markdown(). The title should remain displayed on top regardless of the tab/display the user is viewing*
- *[In the predictive section] Input controls using st.slider(), st.selectbox(), st.number_input(). These controls should be in a sidebar where the user is able to select the model, and a slider for the input features. Ensure the input features are clearly labeled with their sliders. Sliders must be in a comprehensible format (e.g. a slider for co2_log is not as understandable as co2. please make the backend data convert the user's non-log input into the input for regression's depiction). Be careful about feature order and scaling.*
- *The models loaded must be the trained models from the .ipynb file given*
- *[In the predictive section] please display the confidence/prediction intervals alongside point estimates. Use the library MAPIE to wrap the sklearn models and get the prediction intervals for display.*

Along with that create a requirements.txt showing all Python dependencies with version pins formatted like this:

streamlit>=1.31.0
pandas>=2.0.0
numpy>=1.24.0
scikit-learn>=1.4.0
matplotlib>=3.7.0
seaborn>=0.12.0
joblib>=1.3.0

The model should be saved using joblib, like joblib.dump(model_2, 'model.pkl'), and loaded using '@st.cachce_resource'. Plots should use plotly, especially for the interactive visualizations.

Iterate: Please run and test the Streamlit dashboard as if you were a user in a local sandbox. Ensure that the dashboard's writing, labels, buttons, and visuals are clear, readable, and professional. Ensure that the different required sections are accessible on different tabs of the dashboard. This dashboard is to be used by experts and non-experts and must be comprehensive yet approachable. Please include comments explaining any code. The code must be given to me in a format that is ready to paste into a Google Colab notebook for me to test locally and then deploy onto Steamlit live.
Mechanism Check: The regression displays should show the relationships between the various features and the target variable of GDP.
Evaluate: Please ensure the interactive elements work well and seamlessly blend with the visualizations. The dashboard should update in a few seconds for each slider. Use '@st.cachce_resource' to load the dashboard and ensure it does not surpass the 1GB RAM limit.

Let me start by thoroughly reading the notebook to understand exactly what's in it.Now I have everything I need. Let me build the complete dashboard:All models train correctly. Now building the full dashboard:Now let me verify the app logic runs cleanly (syntax + import check):Now create the Colab notebook cell and copy outputs:Here's a full breakdown of what was built and how to use it.

What's in each file

`app.py` — the complete dashboard (~1000 lines). Three tabs:

Tab	Contents
📊 Data Overview	KPI metrics, filterable data table by country/year, global trends area chart, GDP vs CO_2 scatter (bubble size = energy use), correlation heatmap
🔬 Model Explorer	Feature toggles (checkboxes) that retrain all 3 models live, side-by-side actual vs predicted scatter with residual colouring, normalised feature importance comparison across all 3 models, residual distribution histogram
🎯 GDP Predictor	Sidebar sliders in natural units (CO_2 in Mt, not log), auto log-transforms before inference, point estimate + 90/80/95% prediction interval via MAPIE conformal prediction, interval visualised against the observed distribution, 5-nearest-neighbours lookup

`requirements.txt` — pinned versions including `mapie>=0.8.0` and `plotly>=5.18.0`.

`colab_setup.py` — paste each cell sequentially into your notebook.

How to use in Colab

Cell 1 — install dependencies (triggers a runtime restart, normal).

Cell 2 — paste after your existing model-training cells. It saves all 5 `.pkl` files and 2 `.csv` files to the Colab working directory using your already-trained `model_1`, `ridge_model`, `model_2`, and `scaler`.

Cell 3 — paste the full `app.py` content into the triple-quoted string and run to write it to disk.

Cell 4 — starts Streamlit on port 8501 and opens an ngrok tunnel. The printed URL is your live local preview.

For Streamlit Cloud deployment, push all 9 files (app.py, requirements.txt, 5 pkl files, 2 csv files) to a GitHub repo, then connect it at streamlit.io/cloud. The pkl files for this model are small (~5 MB for RF with 200 trees), so no Git LFS needed.

Please reconfigure the files and app.py to be tested locally on my computer instead of through the ipynb file within google colab. I will be using windows to test the streamlit dashboard. please give any files that i will need downloaded.

Good. The `app.py` itself is fine — it uses relative file paths (just filenames, no `/home/claude/` prefixes), so it will work on Windows as-is. What needs to change is: replacing the Colab setup cell with a proper Windows `setup.py` training script, writing a clear README, and packaging everything the user needs to download.Clean. Now write the README:Download all four files. Here's exactly what to do on Windows:

What you're downloading

File	Purpose
`README.md`	Full instructions (also below)
`setup.py`	Trains all 3 models and saves the `.pkl`/`.csv` files — run once
`app.py`	The Streamlit dashboard
`requirements.txt`	All Python dependencies

You also need to **download the raw data** yourself (it's 32 MB, too large to include here):

> Go to `https://github.com/owid/co2-data/blob/master/owid-co2-data.csv` and click **Download raw file** in the top-right corner.

Setup in 4 commands (Windows Command Prompt)

Put all files in one folder, then open a terminal in that folder (`File Explorer → click address bar → type cmd → Enter`):

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

python setup.py

`setup.py` takes 3–5 minutes — the Random Forest with 200 trees is the slow part. When it finishes you'll see the R² scores printed and 7 new files created in your folder. After that, every time you want to run the dashboard:

venv\Scripts\activate
streamlit run app.py

Then open `http://localhost:8501` in your browser. The `setup.py` only needs to run once — the `.pkl` files it creates are permanent.

When booting up app.py locally, I received the following. Does this look good and what is the key error raised? I also noticed there were key errors raised when I used some of the interactive features on the local host.

Two separate issues from the logs:

1. The `use_container_width` warnings — these are harmless deprecation notices from your version of Streamlit. The dashboard still runs fine; I'll fix them in the updated file so the terminal stays clean.

2. The `KeyError: 0` crash on line 951 — this is the real problem. The line `input_values = X_input[0]` is trying to index a pandas DataFrame with `[0]`, which on a DataFrame means "give me the column named `0`" — and no such column exists. Since `_build_feature_row` now returns a DataFrame (not a numpy array), that line needs to use `.iloc[0].values` instead to get the first row as a numpy array.

Let me find every place this pattern appears and fix all of them at once:Two bugs to fix: `X_input[0]` on line 951 and the broadcast subtraction `X_all_arr - X_input` on line 985 (a 2D numpy array minus a DataFrame will also misbehave). Now patch both, plus all the deprecation warnings in one pass:Replace your existing `app.py` with this one. Here's a precise summary of what was wrong and what changed:

Bug 1 — `KeyError: 0` (the crash) — line 951

```
# BEFORE — broken
input_values = X_input[0]

# AFTER — fixed
input_values = X_input.iloc[0].values
```

`X_input` is a pandas DataFrame (one row, 9 columns). Writing `df[0]` on a DataFrame means *"find the column named 0"* — and since no column is named `0`, it throws `KeyError: 0`. This happened on every interaction in Tab 3 that triggered the Random Forest feature contribution chart. `.iloc[0]` selects the first row by position, and `.values` converts it to a plain numpy array for the arithmetic that follows.

Bug 2 — Related broadcasting issue — line 985

```
# BEFORE — would silently misalign or error depending on pandas version
dists = np.linalg.norm(X_all_arr - X_input, axis=1)

# AFTER — fixed
dists = np.linalg.norm(X_all_arr - X_input.values, axis=1)
```

`X_all_arr` is a `(N, 9)` numpy array. Subtracting a DataFrame from a numpy array triggers pandas index-alignment logic rather than clean numpy broadcasting, which can produce NaN-filled results or shape errors. `.values` converts the 1-row DataFrame to a `(1, 9)` numpy array so numpy broadcasts correctly.

Fix 3 — `use_container_width` deprecation warnings (cosmetic)

All 13 instances of `use_container_width=True` replaced with `width="stretch"`, which is the current Streamlit API. Your terminal will now be clean.